

Skip List – Insert Element

–Space Complexity and Algorithm

Program:

```
void insertElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Array to keep track of the path we took (nodes needing pointer
    // updates)
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Start from the highest current level and move down
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    // Move to level 0 and look at the next node
    current = current->forward[0];

    // If the key doesn't already exist, insert it
    if (current == NULL || current->key != key) {
        int rlevel = randomLevel();

        // If the random level is greater than the list's current max
        // level,
        // initialize the update array for the new levels to point to
        // the header.
        if (rlevel > lst->level) {
            for (int i = lst->level + 1; i <= rlevel; i++) {
                update[i] = lst->header;
            }
            lst->level = rlevel;
        }
    }
}
```

```

    // Create the new node and rewire pointers
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
    cout << "Successfully inserted key " << key << endl;
} else {
    cout << "Key " << key << " already exists in the list." << endl;
}
}
int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    struct SkipList *list = createSkipList();
    int choice, key;

    while (true) {

        cout << "\n==== Skip List Menu =====> << endl;
        cout << "1. Insert" << endl;
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Exiting." << endl;
            break;
        }

        switch (choice) {

        case 1:
            cout << "Enter key to insert: ";
            cin >> key;
            insertElement(list, key);
            break;
        }
    }
    return 0;
}

```

Space Complexity Analysis

A. Input Space

The input space is evaluated from two different perspectives: the exact arguments passed to the function, and the overall memory footprint of the entire data structure.

A.1) Parametric Input Space: $O(1)$

The parametric input space strictly measures the memory required to hold the arguments passed directly into the `insertElement` function.

- The function signature is `void insertElement(struct SkipList * lst, int key)`.
- **lst**: A standard memory pointer addressing the existing skip list structure. This takes a small, constant amount of memory (typically 8 bytes on a 64 – bit system).
- **key**: A standard integer variable representing the value to be inserted. This also takes a small, constant amount of memory (typically 4 bytes).
- Because the size of these parameters never scales with the number of elements in the list (n), the parametric input space is strictly $O(1)$.

A.2) Full Data Holistic Input Space: $O(n)$

The holistic input space evaluates the total memory consumed by the entire Skip List data structure containing n elements.

- A standard linked list requires exactly one forward pointer per node. Skip lists add probabilistic express lanes, meaning nodes require arrays of multiple forward pointers.
- However, the probability distribution ensures that the number of pointers drops off exponentially at higher levels.

- *As established in the theoretical analysis, a skip list can be easily configured to require remarkably little extra space – for instance, an average of 1.33 pointers per element when $p = \frac{1}{4}$, or 2 pointers per element when $p = \frac{1}{2}$.*
- *Because the expected maximum number of pointers per node remains a small mathematical constant, the total memory footprint to store n nodes scales directly and linearly with the number of elements i. e. n .*
- *Therefore, the full data holistic input space is $O(n)$.*

B) Auxiliary Space

Auxiliary space measures the temporary, extra memory the algorithm requests specifically to complete the insertion process, which is then destroyed when the function finishes.

- *Inside the insertion module, the primary consumer of auxiliary memory is the update array: `struct Node * update[MAXLEVEL + 1]`.*
- *This array is used to temporarily store the memory addresses of the predecessor nodes at each level of the search path.*

- The array size calculation is explicitly tied to MAX_{LEVEL} .
- In the provided C++ program, MAX_LEVEL is hardcoded to 3, meaning the array allocates exactly 4 slots (32 bytes). In this heavily restricted micro – scenario, the auxiliary space is technically $O(1)$.
- However, in a generic, scalable Skip List mathematically designed to hold n elements, the maximum level is dynamically capped at $H = \log_{\frac{1}{p}} n$. Consequently, the size of the *update* array must scale logarithmically to match the height of the list.

As we have observed in time complexity:

Let n be the total number of elements in the skip list.

Let p be the probability that a node at level k is promoted to level $k + 1$ (typically, 50%, so, $p = 0.5$).

Calculate the Expected Maximum Height(H)

To know how far up we have to climb, we first need to determine the expected maximum height of the entire skip list.

- The probability that any single node reaches level k is p^k .
- Let N_k be the number of nodes at level k . The expected number of nodes at level k across the whole list is:

$$E[N_k] = n \times p^k$$

The maximum height H of the skip list is defined as the level where we expect to find exactly 1 node. Therefore, we set $E[N_k] = 1$:

$$n \times p^H = 1$$

Solving for H :

$$p^H = \frac{1}{n}$$

Putting $\log_p$ at both the side we get:

$$\log_p(p^H) = \log_p\left(\frac{1}{n}\right)$$

$$H = \log_p(n^{-1}) \left[\log_b x^y = y \times \log_b x, \log_b b = 1 \text{ and } \frac{1}{n} = n^{-1} \right]$$

$$H = -1 \times \log_p n \left[\log_b x^y = y \times \log_b x \right]$$

$$H = -\log_p n$$

To get rid of that negative sign, we use the Change of Base Formula $\left(\log_b(x) = \frac{\log_c(x)}{\log_c(b)} \right)$ to change the base from p to $\frac{1}{p}$:

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}(p)} \right)$$

Look at the denominator : $\log_{\frac{1}{p}}(p)$. Because p is the exact

inverse of $\frac{1}{p}$ (meaning $p = \left(\frac{1}{p}\right)^{-1}$), this denominator evaluates

to -1 .

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}\left(\frac{1}{p}\right)^{-1}}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1 \times \log_{\frac{1}{p}}\left(\frac{1}{p}\right)}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1}\right)$$

$$H = \log_{\frac{1}{p}}(n)$$

Alternative calculation:

$$p^H = \frac{1}{n}$$

$$\Rightarrow n = \frac{1}{p^H}$$

We can also write it as:

$$\Rightarrow n = \left(\frac{1}{p}\right)^H \quad [as, 1^H = 1]$$

The fundamental definition of a logarithm is that if $b^y = x$, then $\log_b(x) = y$. Here our base $\left(\frac{1}{p}\right)$, our exponent is H , and our result is n . By simply rewriting it in **log form, we instantly get:**

$$H = \log_{\frac{1}{p}}(n)$$

And

if $p = 0.5$, the expected maximum height is:

$$H = \log_{\frac{1}{0.5}}(n)$$

$$H = \log_{\frac{1}{\frac{1}{2}}}(n)$$

$$H = \log_2(n)$$

Also, in pointer rewiring (Splicing) phase, we have found:

1. Mechanism: The algorithm loops from level 0 up to rlevel, using the saved update[] array to splice the new node into each linked list:

```
for (int i = 0; i <= rlevel; i++) {  
    n->forward[i] = update[i]->forward[i];  
    update[i]->forward[i] = n;  
}
```

A. The Maximum Level Cap:

- *If a node gets exceptionally lucky and flips consecutive Heads, rlevel can grow up to the maximum height of the list:*

$$MAX_LEVEL = \log_{\frac{1}{p}} n$$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n$$

- *In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.*

B. List Level Extension:

- If $rlevel > lst \rightarrow level$, the code runs an extra loop to point new levels to the Header:

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

This is also strictly bounded by $MAX_LEVEL = \log_{\frac{1}{p}} n$

$p = 0.5$, then $MAX_LEVEL = \log_2 n$

There fore ,

Auxiliary Space	Complexity
1.Theoretically	$O(\log_2 n)$
2. Practical Scenario	$O(1)$ $\left[\begin{array}{l} \text{As, Array Size is} \\ \text{Explicitly tied to} \\ \text{Max_Level ,} \\ \text{Which is a} \\ \text{Constant.} \\ \text{(Here , in the program} \\ \text{Max_Level = 3)} \end{array} \right]$

***∴ Total Space Complexity : Full Data Holistic Input + Auxiliary
Space***

In Theoretical Scenario :

$$= O(n) + O(\log_2 n) = O(n) \text{ [As } n \text{ dominates over } \log_2 n \text{]}$$

In practical Scenario:

$$= O(n) + O(1) = O(n) \text{ [As } n \text{ dominates over } 1 \text{]}$$

***Now, this : $O(n)$ is considered as Average or Expected Case of
Total Space Complexity of the Skip List .***

Alternatively Average or Expected

Total Space Complexity of the Skip List , can be calculated as :

***Expected Pointers per Node (Memory Complexity) [Vertical Pointers per Node
Approach – Sums Average
pointers inside a single node.]***

***When a node is created, how many pointers (forward[]) does it
have on average?***

- ***Every node has Level 0: 1 pointer (always, probability = 1).***
- ***It also gets Level 1 , node flips a coin to see if it gets a second pointer with probability: p .***
- ***It also gets Level 2, It flips again for a third pointer with probability: p^2 .***

- It also gets Level 3,, It flips again for a fourth pointer with probability: p^3 , and so on.

The expected total number of pointers in a node is the sum of this infinite geometric series:

$$\mathbb{E}[\text{Pointers per Node}] = 1 + p + p^2 + p^3 + \dots = \sum_{i=0}^{\infty} p^i = \frac{1}{1-p}$$

$$\text{if } p = 0.5 (50\%) = \frac{1}{1-0.5} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2 \text{ pointers per node on average.}$$

And `n` is the total number of elements in the skip list.

Therefore ,n number of elements need : 2n total pointers.

Hence , **Average or Expected Case of Total Space Complexity of the Skip List** : $O(2n) = O(n)$.

Alternative Calculation of Average or Expected

Total Space Complexity of the Skip List , can be calculated as :

Elements per Layer $\left(\begin{array}{c} \text{Horizontal Approach} \\ \text{Sum of Total Nodes on Every floor} \end{array} \right)$

→ **Level 0**: Must contain every single element. Total pointers = n.

→ **Level 1**: Because of the 50% coin flip ($p = 0.5$), only half of the elements

from Level 0 are expected to be promoted. Total expected pointers = $\frac{n}{2}$.

→ **Level 2**: Only half of Level 1 is expected to be promoted. Total expected pointers = $\frac{n}{4}$.

→ **Level 3**: Half of Level 2. Total expected pointers = $\frac{n}{8}$.

$$TotalSpace = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots$$

factor out the n:

$$TotalSpace = n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

We know, through geometric infinite series : $s_{\infty} = \frac{a_1}{1-r}$

$$a_1 = 1 \text{ (First Term) and } r = \frac{\text{Any Term}}{\text{Previous Term}} \Rightarrow \frac{1}{2} = \frac{1}{2} ; \frac{1/4}{1/2} = \frac{1}{2} \text{ etc.}$$

$$\therefore S_{\infty} = \frac{a_1}{1-r} = \frac{1}{1 - \frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2.$$

$$TotalSpace = n \times S_{\infty} = n \times 2 = 2n.$$

Alternatively, Algebraic Shifting Method:

$$\text{Let, } S = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now, Multiplying $\frac{1}{2}$ on both the sides we get:

$$\Rightarrow \frac{1}{2}S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now subtracting we get:

$$\Rightarrow S - \frac{1}{2}S = (1 - 0) + \left(\frac{1}{2} - \frac{1}{2}\right) + \left(\frac{1}{4} - \frac{1}{4}\right) + \left(\frac{1}{8} - \frac{1}{8}\right) + \dots$$

$$\Rightarrow S - \frac{1}{2}S = 1$$

$$\Rightarrow S \left(1 - \frac{1}{2}\right) = 1$$

$$\Rightarrow S \left(\frac{1}{2}\right) = 1$$

$$\Rightarrow S = 2$$

$$\text{TotalSpace} = n \times S = n \times 2 = 2n.$$

***Worst Case of
Total Space Complexity
of the Skip List(Bounded Height)***

Let `n` be the total number of elements in the skip list.

Calculate the Expected Maximum Height(H)

To know how far up we have to climb ,we first need to determine the expected maximum height of the entire skip list.

- *The probability that any single node reaches level `**k**` is **p^k** .*
- *Let `**N<sub>k</sub>**` be the number of nodes at level `**k**` .The expected number of nodes at **level `**k**`** accross the whole list is:*

$$E[N_k] = n \times p^k$$

The maximum height **H** of the skip list is defined as the level where we expect to find exactly **1** node. Therefore, we set $E[N_k] = 1$:

$$n \times p^H = 1$$

Solving for **H**:

$$p^H = \frac{1}{n}$$

Putting $\log_p$ at both the side we get:

$$\log_p(p^H) = \log_p\left(\frac{1}{n}\right)$$

$$H = \log_p(n^{-1}) \left[\log_b x^y = y \times \log_b x, \log_b b = 1 \text{ and } \frac{1}{n} = n^{-1} \right]$$

$$H = -1 \times \log_p n \left[\log_b x^y = y \times \log_b x \right]$$

$$H = -\log_p n$$

To get rid of that negative sign, we use the Change of Base Formula

$\left(\log_b(x) = \frac{\log_c(x)}{\log_c(b)} \right)$ to change the base from **p** to $\frac{1}{p}$:

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}(p)} \right)$$

Look at the denominator : $\log_{\frac{1}{p}}(p)$. Because **p** is the exact

inverse of $\frac{1}{p}$ (meaning $p = \left(\frac{1}{p}\right)^{-1}$), this denominator evaluates

to **-1**.

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}\left(\frac{1}{p}\right)^{-1}}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1 \times \log_{\frac{1}{p}}\left(\frac{1}{p}\right)}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1}\right)$$

$$H = \log_{\frac{1}{p}}(n)$$

Alternative calculation:

$$p^H = \frac{1}{n}$$

$$\Rightarrow n = \frac{1}{p^H}$$

We can also write it as:

$$\Rightarrow n = \left(\frac{1}{p}\right)^H \quad [as, 1^H = 1]$$

The fundamental definition of a logarithm is that if $b^y = x$, then $\log_b(x) = y$. Here our base $\left(\frac{1}{p}\right)$, our exponent is H , and our result is n . By simply rewriting it in **log form, we instantly get:**

$$H = \log_{\frac{1}{p}}(n)$$

if $p = 0.5$, the expected maximum height is:

$$H = \log_{\frac{1}{0.5}}(n)$$

$$H = \log_{\frac{1}{\frac{1}{2}}}(n)$$

$$H = \log_2(n)$$

Also , in pointer rewiring (Splicing) phase , we have found:

1. Mechanism: The algorithm loops from level 0 up to rlevel, using the saved update[] array to splice the new node into each linked list:

```
for (int i = 0; i <= rlevel; i++) {  
    n->forward[i] = update[i]->forward[i];  
    update[i]->forward[i] = n;  
}
```

C. The Maximum Level Cap:

- *If a node gets exceptionally lucky and flips consecutive Heads, rlevel can grow up to the maximum height of the list:*

$$MAX_LEVEL = \log_{\frac{1}{p}} n$$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n$$

- *In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.*

D. List Level Extension:

- If $rlevel > lst \rightarrow level$, the code runs an extra loop to point new levels to the Header:

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

This is also strictly bounded by $MAX_LEVEL = \log_{\frac{1}{p}} n$

$p = 0.5$, then $MAX_LEVEL = \log_2 n$

Even if a node gets exceptionally lucky and flips consecutive Heads, its level is not allowed to grow infinitely; it is capped at the maximum height of the list ($MAX_{LEVEL} = \log_2 n$)

Hence, Worst Case of Total Space Complexity

of the Skip List(Bounded Height) = $n \times \log_2 n = n \log_2 n = O(n \log_2 n)$
or $O(n \log n)$,

Note: Here number of pointers = $n \log_2 n$, where $\log_2 n$ is number of levels, n is number of elements and number of pointers = $n \log_2 n$.

→ Every single node in the list keeps getting promoted to higher and higher levels.

→ To stop the list from taking up infinite memory, the algorithm artificially stops the nodes from growing once they hit the MAX_LEVEL cap,

which mathematically is $\log_2 n$.

→ In this disaster scenario, we do not have a mix of short and tall nodes.

Every single one of our n nodes is perfectly tied at the maximum height of $\log_2 n$.

→ If we have n nodes, and every single one holds $\log_2 n$ pointers, we get $n \times \log_2 n$ total pointers.

*Example: Suppose There are 1024 nodes and $\log_2 1024 = 10$ levels.
 $1024 \times 10 = 10240$ pointers exists.*

And $n < n \log_2 n$ or $n \log_2 n$ dominates over n , Hence, $O(n \log_2 n)$.

Worst Case of Total Space Complexity of the Skip List(UnBounded Height)

If there is absolutely no artificial ceiling programmed into the algorithm, a theoretical coin could flip heads infinitely. A single node would attempt to generate an infinite number of pointers, exhausting all available system memory.

Leads to : $O(\infty)$

This is precisely why all real – world and academic algorithms must enforce

a maximum level cap.

Note : The algorithm requires an **update** array to track the pointer path, which scales with the expected maximum height of the list ($\log_2 n$). However, in practical implementations that enforce a constant upper bound on the maximum level (e. g., 32), the auxiliary space becomes $O(1)$ since the array size is strictly capped to a constant size in real word scenario of a skip list. That is Only the **auxiliary space** used by the **update[]** array becomes $O(1)$. The skip – list nodes themselves still require space that grows with n elements (**Full Data Holistic Input Space**).

In fact, the total expected space for the skip – list structure is typically $O(n)$."

Algorithm: SkipListInsert(LST, KEY)

1. [Initialize Pointers and Arrays]

1. a. **SET CURRENT := HEADER[LST].**

1. b. **Declare an array UPDATE of size MAX_LEVEL + 1 and initialize all elements to NULL.**

2. [Traverse List to Record Path]

2. a. **Repeat for $i = \text{LEVEL}[LST]$ down to 0:**

2. b. **Repeat while $\text{FORWARD}[i][\text{CURRENT}] \neq \text{NULL AND INFO}[\text{FORWARD}[i][\text{CURRENT}]] < \text{KEY}$:**

2. b. i. **SET CURRENT := FORWARD[i][CURRENT].**

[End of while Loop]

2. a. i. **SET UPDATE[i] := CURRENT.**

[End of for Loop]

3. [Move to Target Node]

SET CURRENT := FORWARD[0][CURRENT].

4. [Check for Duplicates]

*If CURRENT = NULL OR INFO[CURRENT] ≠ KEY, Then proceed to Step 5.
Else, proceed to Step 9.*

5. [Generate Random Level]

SET RLEVEL := CALL randomLevel().

6. [Adjust List Max Level]

If RLEVEL > LEVEL[LST], Then:

Repeat for i = LEVEL[LST] + 1 to RLEVEL:

SET UPDATE[i] := HEADER[LST].

[End of for loop]

SET LEVEL[LST] := RLEVEL.

[End of If Structure]

7. [Create Node and Rewire Pointers]

SET NEWPTR := CALL createNode(KEY, RLEVEL).

Repeat for i = 0 to RLEVEL:

SET FORWARD[i][NEWPTR] := FORWARD[i][UPDATE[i]].

SET FORWARD[i][UPDATE[i]] := NEWPTR.

[End of for Loop]

8. [Success Message]

Write: "Successfully inserted key ", KEY.

Proceed to Step 10.

9. [Handle Duplicate Key]

Write: "Key ", KEY, " already exists in the list."

10. [Exit]

END OF ALGORITHM.